JS Problems SDE3

5 min read Updated 20 May 2026 Topic 50 of 77

Chapter 19: JavaScript Problems — SDE3 (5+ Years)

Total Problems: 2 **Difficulty Range:** Senior to Staff **Time per problem:** 45-60 minutes

Why This Chapter Matters

These two problems are system-level implementations that test deep JavaScript knowledge. They're asked at Google, Amazon, and top-tier companies for senior/staff frontend engineers. Each one requires understanding how tools you use daily (bundlers, state management) work under the hood.

19.1 Implement a Simplified Module Bundler

What it does: Takes a JavaScript entry file, resolves all its `import` statements recursively, builds a dependency graph, and outputs a single self-executing bundle.

This is what Webpack/Vite does at its core.

JAVASCRIPT Copy

```
const fs = require('fs');
const path = require('path');
```

```
  // Simple regex-based import extraction (real bundlers use AST parsers)
  const importRegex = /import\s+(?:{[^}]+}|[\w]+)\s+from\s+['"]([^'"]+)['"
  const dependencies = [];
  let match;
  while ((match = importRegex.exec(content)) !== null) {
    dependencies.push(match[1]);
  }

  return { filePath, content, dependencies };
}

// Step 2: Build dependency graph (BFS)
function buildGraph(entryPath) {
  const entry = parseModule(entryPath);
  const graph = [entry];
  const visited = new Set([entryPath]);

  const queue = [entry];
  while (queue.length > 0) {
    const module = queue.shift();
    const dir = path.dirname(module.filePath);

    for (const dep of module.dependencies) {
      const depPath = path.resolve(dir, dep.endsWith('.js') ? dep : dep +
      if (visited.has(depPath)) continue;
      visited.add(depPath);

      const depModule = parseModule(depPath);
      graph.push(depModule);
      queue.push(depModule);
    }
  }

  return graph;
}

// Step 3: Bundle into a single self-executing file
function bundle(entryPath) {
  const graph = buildGraph(entryPath);
```

```
    let code = mod.content
      .replace(/import\s+{([^}]+)}\s+from\s+['"]([^'"]+)['"]/g,
        (_, imports, source) => `const {${imports}} = require('${source}')
      .replace(/export\s+(?:default\s+)?/g, 'module.exports = ');

    return `
      // ${mod.filePath}
      ${index}: function(module, exports, require) {
        ${code}
      }`;
  });

  return `
    (function(modules) {
      const cache = {};

      function require(id) {
        if (cache[id]) return cache[id].exports;
        const module = { exports: {} };
        cache[id] = module;
        modules[id](module, module.exports, require);
        return module.exports;
      }

      require(0); // Start from entry module
    })({${modules.join(',')}});
  `;
}
```

What this teaches:

- Dependency resolution is graph traversal (BFS/DFS)
- Circular dependencies need detection (the `visited` set)
- Module isolation uses closures (each module runs in its own function scope)
- The runtime `require` function is a cache + function call

What real bundlers add: AST parsing (Babel/SWC) instead of regex, tree shaking (remove unused exports), code splitting (multiple output chunks), source maps, CSS/image handling via loaders, and hot module replacement.

19.2 Implement a Reactive State System (Vue-like reactivity)

What it does: When you modify a state object, all functions that depend on that state automatically re-run. This is how Vue 3's reactivity works internally and how libraries like MobX and Solid.js operate.

JAVASCRIPT Copy

```
let activeEffect = null;

class Dep {
  constructor() {
    this.subscribers = new Set();
  }

  track() {
    if (activeEffect) {
      this.subscribers.add(activeEffect);
    }
  }

  notify() {
    this.subscribers.forEach(effect => effect());
  }
}

// Create a reactive proxy around an object
function reactive(obj) {
  const deps = new Map(); // property → Dep

  function getDep(key) {
    if (!deps.has(key)) deps.set(key, new Dep());
    return deps.get(key);
  }
```

```
    get(target, key) {
      getDep(key).track();  // Register the current effect as a subscriber
      const value = target[key];
      // Recursively make nested objects reactive
      if (typeof value === 'object' && value !== null) {
        return reactive(value);
      }
      return value;
    },

    set(target, key, value) {
      const oldValue = target[key];
      target[key] = value;
      if (oldValue !== value) {
        getDep(key).notify();  // Re-run all effects that depend on this p
      }
      return true;
    },
  });
}

// Create a reactive effect (auto-tracks dependencies)
function effect(fn) {
  const wrappedEffect = () => {
    activeEffect = wrappedEffect;
    fn();  // During execution, any property access is tracked
    activeEffect = null;
  };
  wrappedEffect();  // Run once to establish initial dependencies
  return wrappedEffect;
}

// Computed value (derived state)
function computed(fn) {
  let cached;
  let dirty = true;

  const computedEffect = effect(() => {
    cached = fn();
    dirty = false;
  });
```

```
      return cached;
    },
  };
}

// ── Usage ──────────────────────────────────────────────

const state = reactive({
  firstName: 'Ankit',
  lastName: 'Kumar',
  age: 28,
  address: { city: 'Bangalore' },
});

// This effect auto-tracks: it depends on firstName and lastName
effect(() => {
  console.log(`Full name: ${state.firstName} ${state.lastName}`);
});
// Logs: "Full name: Ankit Kumar"

state.firstName = 'Rahul';
// Automatically logs: "Full name: Rahul Kumar"

state.age = 29;
// Nothing logs — the effect doesn't depend on age

// Nested reactivity
effect(() => {
  console.log(`City: ${state.address.city}`);
});
// Logs: "City: Bangalore"

state.address.city = 'Mumbai';
// Automatically logs: "City: Mumbai"
```

What this teaches:

- **Dependency tracking:** The `Proxy` `get` trap records which effect is reading which property

- **No manual subscriptions:** Unlike Redux (dispatch → subscribe), reactive systems track dependencies automatically just by reading properties
- **This is how Vue 3, MobX, and Solid.js work under the hood**

What production reactive systems add: Batch updates (don't re-run effects synchronously on every change), scheduler (queue effects and flush at the right time), shallow vs deep reactivity, readonly proxies, and handling array mutations (push, pop, splice need special handling because they don't trigger `set` for the length property).

Interview approach: Start with `reactive` + `effect`. If time permits, add `computed`. Explain the Proxy-based dependency tracking clearly. Mention that Vue 2 used `Object.defineProperty` (couldn't detect new property additions or array index mutations) and Vue 3 switched to Proxy (handles everything).

QUICK CHECK — no marks, just for you.

1. This is the lesson's Proxy-based reactivity system, trimmed to a single level. What does it log?

JAVASCRIPT Copy

```
let activeEffect = null;

class Dep {
  constructor() { this.subscribers = new Set(); }
  track() { if (activeEffect) this.subscribers.add(activeEffect); }
  notify() { this.subscribers.forEach(effect => effect()); }
}

function reactive(obj) {
  const deps = new Map();
  function getDep(key) {
    if (!deps.has(key)) deps.set(key, new Dep());
    return deps.get(key);
  }
  return new Proxy(obj, {
    get(target, key) { getDep(key).track(); return target[key]; },
```

```
      if (oldValue !== value) getDep(key).notify();
      return true;
    },
  });
}

function effect(fn) {
  const wrapped = () => { activeEffect = wrapped; fn(); activeEffect = null;
  wrapped();
  return wrapped;
}

const state = reactive({ count: 1, other: 9 });

effect(() => { console.log(`count: ${state.count}`); });

state.other = 10;
state.count = 2;
state.count = 2;
```

- ○ count: 1
count: 2
- ○ count: 1
count: 2
count: 2
- ○ count: 1
count: 1
count: 2
count: 2
- ○ count: 2

Check answer

2. In the simplified module bundler, how does buildGraph discover every module that must end up in the bundle, starting from the entry file?

It reads a manifest of dependencies that the developer lists in a config file

- It relies on Node's require.cache to report which files were loaded at runtime
- It scans the whole project directory and includes every .js file it finds
- It traverses the dependency graph from the entry file, parsing each module's imports and queueing the newly resolved files

Check answer

3. According to the lesson, what distinguishes a reactive system like Vue 3 or MobX from Redux in terms of how something comes to depend on a piece of state?

- The reactive system requires each effect to declare a dependency array up front, like a hook
- The reactive system re-runs every registered effect on any state change, so dependencies do not matter
- The reactive system records the dependency automatically when a property is read, so no explicit subscription is needed
- Both track dependencies automatically; the difference is only that Redux stores state immutably

Check answer

ARTICLES & DOCS

Frontend System Desi...

Tricky JS questions (repo)

github.com

VIDEOS



JS questions for senior developers

YouTube



Advanced JS interview questions

YouTube

Top advanced JS interview questions

YouTube

Hand-picked references for this lesson.